

Architectural Blueprint for an Edge-Computed Multimodal AI Mathematical Tutoring Canvas

The engineering of a fully localized, multimodal artificial intelligence study companion represents a profound intersection of edge-computing machine learning, high-frequency human-computer interaction, and cognitive pedagogy. Traditional digital mathematics environments rely heavily on monolithic cloud architectures, subjecting sensitive student data to external processing and introducing pedagogical friction via network latency. Conversely, constructing a closed-loop learning environment designed to operate entirely on consumer-grade local hardware—specifically constrained to hardware profiles such as the NVIDIA RTX 5060 GPU with 8 gigabytes of VRAM—demands extreme architectural optimization¹. Furthermore, for such an intelligent tutoring system to function autonomously and "perfectly" as an educational tool, it must be underpinned by a robust curriculum ingestion pipeline capable of dynamically generating and evaluating mathematical questions³.

This report provides an exhaustive, low-level technical specification for deploying a multimodal mathematical canvas equipped with Optical Character Recognition (OCR), algorithmic hesitation detection, abstract syntax tree (AST) validation, a procedural question generation engine, and a local Vision-Language Model (VLM) for Socratic intervention. By synthesizing real-time biometric telemetry with advanced spatial parsing and the Free Spaced Repetition Scheduler (FSRS), the architecture establishes a deterministic, privacy-first educational flywheel capable of resolving complex, multi-line mathematical reasoning¹.

1. Digital Whiteboard Orchestration and Kinematic Telemetry

The interactive digital whiteboard constitutes the primary data ingestion plane. Its structural integrity and capacity to expose low-level spatio-temporal metrics are foundational to the downstream machine learning pipelines. Because the application continuously captures mathematical equations to calculate cognitive load and hesitation markers, the canvas must process thousands of discrete vector coordinates per minute without inducing visual latency or degrading the single-threaded rendering cycle of the frontend application.

1.1 Rendering Paradigms: Immediate vs. Retained Mode Frameworks

When engineering a high-fidelity drawing interface within a React ecosystem, the selection of the underlying graphics API profoundly impacts rendering performance and serialization speed. The primary options involve Document Object Model (DOM) node-based editors, Scalable Vector Graphics (SVG) wrappers, and immediate-mode HTML5 Canvas frameworks. DOM-based libraries offer highly intuitive state-driven UI development but experience catastrophic performance degradation when scaling beyond a few hundred nodes due to browser memory overhead and continuous layout reflows¹. Conversely, raw HTML5 canvas

rendering provides immediate-mode processing, meaning shapes are drawn pixel-by-pixel, and the browser does not retain an interactive object model in memory. This guarantees continuous high frame rates even with thousands of active stroke paths, making it ideal for freehand mathematical sketching¹.

The integration of the react-sketch-canvas library provides an optimal abstraction over HTML5 canvas primitives for this specific domain¹. A rigorous architectural comparison elucidates why alternative ecosystem libraries, such as Fabric.js and Konva.js, are sub-optimal for raw mathematical transcription.

Framework	Rendering Mechanism	Interactive Object Model	Architectural Suitability for Mathematical Transcription
react-sketch-canvas	HTML5 Canvas / SVG paths	Read-only path extraction	Excellent for freehand stroke capturing and programmatic path serialization natively within React. Exposes native exportPaths() and exportImage() methods.
Fabric.js	HTML5 Canvas with SVG Parsing	Fully retained objects	Ideal for complex object transformations and layered image manipulation, but adds unnecessary overhead for raw freehand math strokes.
Konva.js	Immediate Mode HTML5 Canvas	Custom node hierarchy	Superior for complex scene graphs and isolated layer management; structurally oversized for single-layer

			freehand sketching.
Raw HTML5 Canvas	Immediate Mode	None	Requires extensive custom state management, event listeners, and manual undo/redo array maintenance.

The react-sketch-canvas framework encapsulates undo/redo stack management, SVG serialization, and stroke timing directly within its API contract, enabling the extraction of both temporal stroke data and static base64 image snapshots without heavy dependency bloat¹. By maintaining an internal coordinate system bounded by the user's active viewport, the application can programmatically crop the resulting base64 payload to only encompass the most recently drawn strokes, dramatically reducing the dimensionality and HTTP transport weight required by the downstream OCR models¹.

1.2 Un-Throttling Browser Events for Granular Kinematics

The mathematical problem-solving process is inherently temporal. The velocity of handwritten strokes and the spatial gaps between individual characters provide critical physiological indicators of cognitive load, confidence, and hesitation¹. High-velocity, continuous strokes generally correlate with mechanical transcription or confident problem-solving, whereas frequent lifting of the stylus, varying stroke pressure, and highly localized spatial pausing suggest acute hesitation¹.

Capturing this kinematic data requires bypassing standard browser event limitations. Standard Document Object Model event listeners, such as pointermove, are notoriously insufficient for high-fidelity velocity calculations because modern browser engines threshold and coalesce UI events to align with the monitor's display refresh rate (typically locking event dispatching to 60Hz or 120Hz to preserve battery life and rendering performance)¹.

To capture intermediate hardware polling states, the architecture leverages the native PointerEvent.getCoalescedEvents() API¹. This method retrieves an un-throttled sequence of all discrete pointer events that occurred between standard animation frames. By iterating through the coalesced events array within a custom hook (useStrokeAnalytics.ts), the frontend constructs a dense matrix of timestamped spatial coordinates¹. Instantaneous velocity between any two coordinates is derived via the Euclidean distance formula divided by the delta in their respective timestamps:

$$v = \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{t_2 - t_1}$$

This high-frequency temporal data is bundled into a `stroke_metrics_json` payload and continuously evaluated by the client-side state manager to calculate drawing rhythms, ultimately writing this data to `DSC_Kinematics.csv` during an intervention to build a predictive model of student frustration³.

1.3 Multimodal Sensor Fusion and the Stuckness Heuristic

Kinematic data from the canvas represents only one vector of the user's cognitive state. To trigger intelligent tutoring interventions asynchronously without requiring the user to press a "help" button, the system calculates a unified "Stuckness Score" by fusing stroke data with concurrent multimodal streams¹.

WebGazer.js operates asynchronously within the browser, utilizing the local webcam to project a continuous stream of gaze coordinates onto the viewport. By mapping these spatial coordinates onto the internal coordinate system of the canvas, the application tracks visual fixations. A prolonged fixation on a recently completed matrix, coupled with a cessation of drawing activity, serves as a strong heuristic indicator that the user is verifying logic or experiencing confusion¹. Concurrently, backend ESP32 biometric UDP telemetry processes raw Electroencephalography (EEG) buffers via a fast Fourier transform (FFT) to extract alpha, beta, and gamma frequency bands¹. Elevated gamma band activity indicates highly active cognitive processing, providing a biological proxy for cognitive load.

The deterministic mathematical model for the Stuckness Score is heavily parameterized against idle time, eraser events, and biometric triggers, executed via the `compute_stuckness` function in `intervention_handler.py`⁶. The algorithm calculates a penalty score ranging from 0.0 to 1.0:

$$S = 0.4 \times \min\left(\frac{t_{idle}}{90}, 1\right) + 0.3 \times \min\left(\frac{e_{count}}{5}, 1\right) + 0.3 \times \min\left(\max\left(\frac{\gamma - 55}{30}, 0\right), 1\right)$$

If an OCR parse is available and determines that a mathematical step is incomplete (`ocr.incomplete_step`) or missing a valid LaTeX string entirely, the base score is artificially nudged upwards by an additional 0.05 per condition⁶. When the score surpasses a strict

threshold ($S \geq 0.50$), the system categorizes the learner as actively stuck and triggers an asynchronous canvas snapshot sequence, transmitting a `MathInterventionIn` payload containing the canvas image, path data, and idle metrics to the backend server³.

2. Spatial Segmentation and Mathematical Optical Character Recognition

Once the FastAPI backend receives the localized base64 canvas snapshot via the `POST /api/math/ocr` or `POST /api/math/intervention` endpoints, the system executes a complex transformation of raw pixel data into a semantic LaTeX representation³. Bounded by the 8 gigabytes of VRAM on the target RTX 5060, the architecture decouples spatial layout parsing from mathematical recognition to preserve memory¹.

2.1 Morphological Pre-Processing via OpenCV

Handwritten digital mathematics introduces immense spatial entropy. Strokes frequently overlap, baselines slant unpredictably, and microscopic gaps exist in rapidly drawn symbols (such as an equals sign or a division operator). A pure machine learning approach fails catastrophically if the geometric layout is entirely disjointed¹.

To counteract this, the backend integrates a deterministic computer vision segmentation pre-processing layer using OpenCV prior to passing the data to the neural networks¹. The base64 image is converted into a grayscale NumPy matrix, and morphological transformations—specifically dilation and erosion—are applied. Dilation expands the boundaries of the foreground pixels, artificially bridging microscopic gaps between temporally related strokes¹. The system then utilizes connected components analysis to identify independent clusters of pixels, isolating distinct mathematical steps.

If a student heavily scribbles over an error to erase it, the frontend's dense overlapping stroke path data is cross-referenced with the OpenCV output. Regions exhibiting extreme pixel density and chaotic stroke intersections are algorithmically masked out, preventing the downstream OCR model from hallucinating complex syntax from meaningless scribbles¹.

2.2 Multi-Line Segmentation via Pix2Text MFD

Attempting to parse an entire multi-line mathematical proof in a single inference pass degrades accuracy, as sequence-to-sequence models frequently attempt to merge distinct problems into a single massive equation⁸. Traditional approaches to mitigate this involved forcing users to write in a restrictive "3-band" UI (e.g., exponents in a top band, numerators in a middle band, denominators in a bottom band)². However, this introduces immense cognitive friction and violates natural freehand workflows.

Instead, the architecture utilizes the Pix2Text (P2T) Mathematical Formula Detection (MFD) model for structural layout parsing². Operating on a heavily optimized YOLOv7 backbone (derived from the CnSTD framework), the Pix2Text MFD ONNX model is exceptionally compact—approximately 80.3 MB—making it blazing fast for spatial line detection prior to full transcription². When the student finishes a line, the MFD model scans the OpenCV-processed canvas, drawing distinct bounding boxes around isolated equations and detecting the two-dimensional layout of the whiteboard². This isolated bounding box is then cropped and dispatched sequentially to the primary recognition engine, effectively parsing complex multi-line mathematical logic without restricting the user's natural spatial mapping².

2.3 Mathematical Formula Recognition via TexTeller 3

For the direct transcription of cropped bounding boxes into LaTeX, the architecture relies on TexTeller 3². TexTeller 3 utilizes a Vision Transformer (ViT) architecture trained on an exhaustive dataset of 80 million image-formula pairs, granting it exceptional generalization capabilities over complex multi-line formulas, matrices, and mixed alphanumeric text².

The choice of TexTeller 3 over alternative models, such as Microsoft's TrOCR, is rooted in dimensional capability. While TrOCR achieves state-of-the-art results on one-dimensional text sequences, it scores 0% on standard handwritten mathematical benchmarks out of the box because its attention mechanisms fail to capture two-dimensional spatial hierarchies (such as

nested fractions, matrices, or limits)². Conversely, TexTeller is explicitly designed to recognize printed and handwritten 2D formulas in a single forward pass, natively parsing complex structures like the `\begin{align}` environment².

2.4 Hardware Acceleration and ONNX Runtime Profiling

To achieve end-to-end execution latencies below 1.5 seconds on consumer hardware, the inference engines must be meticulously tuned. Early iterations of the codebase suffered from severe GPU underutilization, as the `texteller_onnx.py` and `mfd_onnx.py` modules hardcoded the ONNX Runtime execution provider to CPU-only (`providers=["CPUExecutionProvider"]`)⁶. This artificial constraint forced the local RTX 5060 to sit idle while the host processor struggled with intensive Vision Transformer matrix multiplications⁶.

The architectural correction replaces the hardcoded string with an automatic provider resolution array: `providers=["CUDAExecutionProvider", "CPUExecutionProvider"]`⁶. This configuration guarantees that ONNX Runtime attempts to load the computation graph onto the GPU's VRAM via CUDA first, gracefully falling back to the CPU only if CUDA drivers are misconfigured or VRAM is fully exhausted by the LLM⁶.

Furthermore, optimizing graph execution via the `onnxruntime.SessionOptions()` interface is absolutely critical for latency stabilization. Setting `execution_mode = ort.ExecutionMode.ORT_SEQUENTIAL` instructs the runtime to execute graph operators sequentially rather than attempting parallel execution¹³. While seemingly counterintuitive, sequential execution dramatically reduces thread contention and context-switching overhead for highly linear Transformer models running alongside asynchronous I/O tasks¹⁴.

Correspondingly, `intra_op_num_threads` must be tightly bounded to a conservative integer (e.g., 2 or 4) rather than allowing ONNX to aggressively consume all available logical cores¹⁴. If OpenMP is enabled on the system, setting the environment variable `OMP_WAIT_POLICY=PASSIVE` (throughput mode) ensures that threads yield the CPU after finishing their current task, rather than utilizing active spin-waiting (ACTIVE mode) which artificially inflates CPU usage and starves the FastAPI event loop¹⁴.

ONNX Runtime Optimization Variable	Configured Value	Architectural Justification
Execution Providers	<code>["CUDAExecutionProvider", "CPUExecutionProvider"]</code>	Enables RTX 5060 GPU acceleration with a failsafe CPU fallback to prevent runtime application crashes ⁶ .
<code>execution_mode</code>	<code>ORT_SEQUENTIAL</code>	Minimizes thread contention for graph nodes that do not contain highly

		branched parallel operations, stabilizing latency variance ¹³ .
intra_op_num_threads	2 or 4	Caps thread pooling, preventing thread starvation for concurrent FastAPI asynchronous tasks ¹⁴ .
graph_optimization_level	ORT_ENABLE_ALL	Applies constant folding, node fusion, and layout optimizations natively at load time ¹⁴ .
enable_profiling	True (During debugging)	Generates JSON trace files compatible with chrome://tracing to precisely identify operator-level latency bottlenecks ¹³ .

2.5 Bimodal Fusion, Fallback Tiers, and Structural Verification

A persistent vulnerability in Vision-Language Models and ViT-based architectures is the propensity for algorithmic hallucination; the model may generate mathematically valid but visually disconnected LaTeX when faced with highly ambiguous strokes². To prevent corrupted data from entering the pedagogical loop, the system implements a `structure_verify.py` pipeline that compares geometric stroke properties against the semantic properties of the generated LaTeX⁵.

However, relying strictly on hand-tuned geometric heuristics (e.g., assuming a superscript must

have a height $< 0.7 \times$ median height and sit $> 0.25 \times$ above the baseline) introduces extreme fragility. This heuristic quietly breaks the "stay silent when unsure" design on single symbols or unusually proportioned handwriting, causing the `structural_confidence` metric to misfire and inadvertently suppress valid tutoring⁶.

To resolve this structurally, the architecture implements a multi-tiered fallback and validation strategy. Advanced paradigms in the field, such as Uni-MuMER, demonstrate that forcing the neural network to learn fundamental properties of mathematics via auxiliary tasks—such as Tree-Aware Chain-of-Thought (Tree-CoT) and Symbol Counting—dramatically reduces critical mathematical mistranslations⁸. If the primary TextTeller pass yields low confidence, the pipeline evaluates the raw stroke sequence through an online temporal representation⁶. Because the

temporal model relies on stroke order and velocity sequences ($\Delta x, \Delta y$, `pen_up`) rather than

static pixels, it is entirely immune to heavy ink overlap⁶.

If the internal confidence remains below acceptable thresholds, the system invokes the OLLAMA_VISION_MODEL (e.g., LLaVA) or the remote NIM API teacher endpoint (backend.integrations.nim_client) as a final sequential rescue tier before gracefully failing and requesting manual human clarification³.

3. Procedural Question Generation and Curriculum Ingestion

An intelligent tutoring system cannot function effectively as a static OCR tool; it requires a structured curriculum and an infinite supply of practice variations to guide the student toward mastery³. To satisfy the requirement for the system to "work perfectly," the backend implements a highly robust question generation and ingestion pipeline, orchestrated through the backend/math/router.py API endpoints³.

3.1 Layer-0 Procedural Drills and SymPy Generators

Storing every possible mathematical question in a relational database is fundamentally unscalable. Instead, the architecture utilizes procedural generation via the generators/layer0.py module⁵. Guided by a skills.json dependency tree, the application maps out a hierarchical curriculum ranging from basic arithmetic to advanced calculus³.

When a student requests a new practice session via the GET /api/math/skills endpoint, the next_available_node algorithm traverses the skill tree, analyzing the user's historical performance³. Upon selecting an appropriate skill tag (e.g., algebra.distributive_property), the backend invokes a parameterized Python generator. Utilizing the SymPy library, the generator

establishes a generalized mathematical template (e.g., $a(bx + c)$) and injects randomized integer coefficients and variable names².

Because the generator relies on SymPy's algebraic engine, it simultaneously computes the precise mathematical solution and formats the output into pristine LaTeX. This ensures an infinite supply of mathematically sound practice questions, eliminating the possibility of a student memorizing a specific answer key².

3.2 Question Bank Import and Export Pipeline

While procedural generation handles standard skill drills, educators and administrators often require the ingestion of highly specific, context-rich word problems or standardized test formats. The system supports bulk ingestion via the POST /api/math/questions/import/json and POST /api/math/questions/import/file endpoints³.

The parse_import_payload function validates incoming JSON structures against a strict Pydantic schema (MathImportBundle), ensuring that every question contains a defined topic, a textual prompt, and an expected answer format³. To prevent database corruption during mass ingestion, administrators can utilize the POST /api/math/questions/import/preview endpoint, which performs a dry-run validation of the payload, returning a truncated preview of valid items and explicit error messages for malformed entries without writing to the SQLite database³.

Successfully imported questions are managed via the math_questions table, allowing for

sophisticated querying by topic, difficulty, and active status³.

3.3 Adaptive Evaluation and Equivalence Checking

Evaluating a student's final answer poses a complex algorithmic challenge. In mathematics, $\frac{x}{2}$

and $0.5x$ are algebraically equivalent, but a naive string comparison will mark the student's submission as incorrect, causing immense frustration.

When a student submits their final whiteboard state for grading (POST /api/quiz/{id}/answer or POST /api/math/eval), the system isolates the recognized LaTeX string and executes an equivalence check via the eval_expression function within eval_service.py³. The backend parses both the expected answer (from the database or procedural generator) and the student's submitted answer into SymPy expressions. The algorithm subtracts the student's expression from the expected expression and applies the simplify() method. If the simplified result evaluates exactly to zero, the student's answer is marked mathematically correct, regardless of the specific formatting or ordering of the algebraic terms².

4. Abstract Syntax Tree Validation and SymPy Parsing

Transforming pixels into LaTeX is insufficient for automated tutoring; the application must computationally verify the mathematical logic. A student may write an incomplete step (e.g., $\frac{2x+1}{}$, pausing before completing the denominator), which traditional OCR pipelines will hallucinate into a complete, albeit mathematically impossible, expression².

4.1 The SymPy Parsing Architecture and ANTLR4 Conflicts

To evaluate the mathematical validity of the OCR output before providing a hint, the FastAPI backend routes the generated LaTeX string through an algebraic parsing tree using the sympy.parsing.latex module, which converts it into a rigid Abstract Syntax Tree (AST)². This structural validation ensures all brackets match, operators are balanced, and variables are logically positioned.

Deploying this within a modern Python 3.14 environment presents a severe architectural hurdle. The SymPy LaTeX parser historically relies on an ANTLR4-generated grammar¹⁸. The antlr4-python3-runtime is notoriously fragile regarding Python versioning, tightly binding itself to version 4.10 or 4.11¹⁹. When invoked in Python 3.14, the runtime successfully installs but silently fails during execution, rendering internal objects like LaTeXLexer as None and throwing catastrophic TypeError: 'NoneType' object is not callable exceptions²¹. Furthermore, parsing malformed nested groups occasionally bypasses standard exception handling, trapping the backend in an infinite RecursionError².

The mitigation strategy involves transitioning the backend dependency away from the deprecated ANTLR4 parser toward SymPy's experimental Lark-based backend, which natively supports modern Python distributions without relying on brittle runtime deserialization¹⁸. Furthermore, the parser invocation is wrapped in a strict asynchronous try/except block equipped with a bounded computational timeout. If the parser enters a recursive loop or throws a parsing exception, the backend cleanly catches the error and flags the

incomplete_step boolean².

4.2 Preserving Structural State with evaluate=False

When the system parses a string, the default behavior of algebra engines is to immediately

simplify the expression. If a student writes $2x + 3x$, standard evaluation automatically

reduces this to $5x$. However, an intelligent tutoring system must analyze the exact structural format the student wrote to deduce their current logical process and identify specific algebraic misconceptions².

To achieve this, the system explicitly passes the parameter `evaluate=False` during the `parse_latex` invocation². This critical configuration suppresses automatic simplification, freezing the AST in its raw state²⁰. This allows the AI tutor to observe that the user successfully combined terms on the board without the parser silently masking the intermediate step. If an expression fails AST parsing entirely, the system intercepts the error and emits the `incomplete_step` signal, prompting the tutor to output a structural scaffolding nudge rather than generating an unhelpful computational error².

5. Deterministic VLM Orchestration and Socratic Pedagogy

The ultimate objective of the edge-computed core is not to act as an automated calculator, but as a pedagogical agent. When the "Stuckness Score" triggers an inference cycle via the `POST /api/math/intervention` route, the backend synthesizes the raw canvas image, the generated LaTeX, and the AST validation status into an enriched prompt³. This payload is routed to a local Vision-Language Model (VLM), executing via Ollama, to deliver contextual hints without explicitly revealing the mathematical solution¹.

5.1 Pydantic Schema Enforcement and Logit Masking

A primary failure mode of localized language models operating under 10 billion parameters is their inability to consistently return strictly formatted structured data. Small models frequently append conversational filler or misalign JSON arrays, which instantly breaks the downstream serialization required to render the frontend UI¹.

The architecture definitively solves this via Ollama's native structured output capabilities, tightly integrated with Pydantic serialization¹. The FastAPI backend defines a strict schema dictating the expected output:

```
Python
from pydantic import BaseModel, Field

class SocraticIntervention(BaseModel):
```

```

mathematical_step_identified: str = Field(description="The exact theorem or rule the user is attempting.")
point_of_confusion: str = Field(description="A deduction of where the logic is failing.")
socratic_hint_text: str = Field(description="An open-ended question guiding the user.")
confidence_score: float = Field(description="Model confidence between 0.0 and 1.0.")

```

By passing this schema directly to the Ollama Python client, the underlying llama.cpp engine executes grammar-based constraint generation¹. During token generation, the model's logits are dynamically masked; if the model attempts to predict a token that violates the JSON structure defined by the Pydantic schema, that token's probability is forced to absolute zero¹. This ensures a 100% deterministic output structure, completely eliminating the risk of client-side JSON parsing exceptions and maintaining a fluid educational loop¹.

5.2 Socratic System Prompting and Chain-of-Thought Guardrails

Inducing genuine Socratic behavior requires forcing the model out of its inherent "sycophancy" (the tendency to immediately solve the user's problem)¹. The system prompt structurally mandates absolute behavioral constraints: the model must never generate the final answer, provide the next algebraic step directly, or output LaTeX code representing the solution¹. Furthermore, the structure of the Pydantic schema actively enforces Chain-of-Thought (CoT) reasoning. By requiring the model to generate the `mathematical_step_identified` and `point_of_confusion` fields *before* the `socratic_hint_text`, the algorithm is forced to articulate its diagnostic logic sequentially¹. This internal deliberation maps out the pedagogical intent in text before formulating the question, dramatically improving the relevance and accuracy of the final hint presented to the student¹.

A secondary layer of verification occurs in `intervention_handler.py`. Before the hint is transmitted to the client, the `_hint_passes_socratic_check` function runs a regular expression analysis against the generated text, scanning for specific answer-revealing patterns (e.g., `x = \d` or `solution:)`⁶. If the VLM violates pedagogical directives and generates the answer, the regex trap catches it and suppresses the output, falling back to a safe, rule-based heuristic defined in `_latex_aware_rule_hint`⁶.

5.3 Tutor Silence and Intervention Suppression

To preserve the student's concentration, the tutor must remain completely silent if the underlying OCR data is deemed unreliable. The `build_intervention` function actively evaluates both the overall OCR confidence and the `structural_confidence` metric against predefined constants (`OCR_CONFIDENCE_SILENCE_THRESHOLD = 0.45` and `STRUCTURAL_SILENCE_THRESHOLD`)⁶.

If either metric falls below the threshold, the `tutor_silent` boolean is flagged as `True`⁶. When this occurs, even if the student is actively struggling, the backend completely aborts the LLM generation cycle. Instead, it emits a soft confirmation prompt (e.g., "I couldn't read your last step clearly — show me that line again?") to gracefully prompt the user for clarification without interrupting their thought process with a mathematically incorrect deduction⁶.

5.4 Context Window Management and VRAM Residency

Maintaining a fluid conversational state across a prolonged study session without overloading the 8GB VRAM constraint of the RTX 5060 requires aggressive context window pruning¹. If the backend transmits the entire conversation history—including multiple sequential base64 images—during every subsequent intervention, the context window balloons exponentially, leading to catastrophic out-of-memory (OOM) exceptions¹.

The FastAPI backend implements a strict history limitation protocol. Only the text of the previous Socratic hints and the LaTeX text-based interpretations of prior canvas states are appended to the prompt array. The heavy base64 visual matrices from prior steps are aggressively discarded from the message history once processed¹.

Furthermore, cold-starting a 4.5GB to 7GB VLM from a solid-state drive into GPU VRAM imposes a 3 to 5-second latency penalty. To circumvent this during active study sessions, the architecture leverages the `keep_alive` parameter¹. Setting `keep_alive="10m"` explicitly instructs the Ollama engine to maintain the VLM resident in VRAM for ten minutes after the most recent inference, allowing sequential Socratic hints to generate in milliseconds¹. Upon the conclusion of a Pomodoro cycle, a discrete API call with `keep_alive="0"` explicitly flushes the VRAM, freeing system resources for background processing¹.

6. Mastery Learning and the Spaced Repetition Flywheel

An intelligent tutoring system is incomplete if it fails to convert immediate algorithmic remediation into long-term knowledge retention. When a student struggles—evidenced by incorrect algebraic steps, repeated AI hints, or triggering the Stuckness Score—these specific conceptual gaps must be captured and systematically remediated².

6.1 FSRS vs. Bayesian Knowledge Tracing

Traditional educational technology platforms rely on Bayesian Knowledge Tracing (BKT) to model the probability of a student transitioning from an "unlearned" to a "learned" state². However, BKT fundamentally fails to model the psychological forgetting curve; it assumes that once mastery is achieved, it is permanent².

The architecture abandons BKT in favor of the Free Spaced Repetition Scheduler (FSRS). FSRS utilizes the Difficulty, Stability, and Retrievability (DSR) model to predict memory decay mathematically².

- **Retrievability (R)**: The probability of successfully recalling a concept, which decays exponentially over time according to the formula $R = 0.9^{\frac{t}{S}}$, where t is the elapsed time in days².
- **Stability (S)**: The time required for Retrievability to drop to exactly 90%, representing the core strength of the encoded memory².
- **Difficulty (D)**: A clamped variable ranging from 1 to 10 that models the inherent complexity of the material, which scales the rate at which Stability increases following a

successful review².

Because FSRS explicitly computes temporal memory decay and optimizes the scheduling intervals based on massive historical datasets, it prevents over-testing and schedules reviews with exceptional efficiency, demonstrating a 20-30% reduction in review load compared to legacy algorithms². The FastAPI backend seamlessly integrates this logic utilizing the pure Python package `py-fsrs`, establishing a localized SQLite database that maps unique mathematical skills directly to the review queue².

6.2 Autonomous Grade Mapping

In a traditional SRS system (such as Anki), the user must manually grade their own recall (e.g., clicking "Hard", "Good", or "Easy"). This breaks the immersion of a live tutoring environment. The `srs_bridge.py` module autonomously evaluates the student's performance and maps the severity of the AI intervention directly to an FSRS grade (1-4) without requiring manual input². When an intervention is triggered and the student struggles, the `enqueue_math_review` function is automatically invoked with `correct=False`³.

- **Grade 1 (Again):** The student triggers maximum stuckness, abandons the problem entirely, or forces the AI to reveal the solution. FSRS computes a massive stability drop, updating $S \leftarrow S_{min}$ and resetting the learning phase².
- **Grade 2 (Hard):** The student makes a critical error, the AI provides a specific algorithmic nudge, and the student eventually resolves the error with heavy scaffolding.
- **Grade 3 (Good):** The student completes the step slowly or requires only a minor Socratic hint (e.g., "Check your negative signs").
- **Grade 4 (Easy):** The student breezes through the complex multi-line problem flawlessly without triggering the stuckness heuristic².

If a student receives a hint but subsequently recovers and solves the problem independently, the frontend dispatches a `PATCH /api/math/intervention/{snapshot_id}/recover` request, which gracefully updates the intervention status to "recovered" and schedules a positive FSRS success modifier, rewarding the student for their resilience³.

7. Continual Learning, Telemetry Persistence, and Retraining Loops

A foundational pillar of this privacy-first architecture is the localized retention of analytical data to establish a clean, continuous dataset for future Phase 4 machine learning optimization and OCR fine-tuning¹. Storing high-frequency telemetry and base64 images within a relational SQLite database causes severe fragmentation and degrades read/write throughput. Therefore, the architecture relies on highly optimized, append-only CSV files¹.

7.1 Localized Data Strategy

All user behaviors, interaction states, kinematic metrics, and intervention metadata are continuously buffered in memory on the backend and flushed periodically to standard CSV files prefixed with `DSC_` (Data Science Capture)¹.

- `DSC_Kinematics.csv`: Logs timestamped stroke velocities, path coordinates, and localized

canvas hesitation metrics collected from the useStrokeAnalytics hook¹.

- DSC_Interventions.csv: Managed via intervention_log.py, this file logs the timestamp of the triggered AI intervention, the generated JSON payload (including the Socratic hint), and a boolean value representing whether the learner successfully recovered¹.
- DSC_handwriting_dataset.csv: Specifically targets the local retraining loop. Managed via training_log.py, every time a user manually confirms a correct LaTeX transcription or corrects a hallucinated parse, the POST /api/math/train/sample endpoint records the cropped PNG path, the confirmed LaTeX string, and the raw paths³.

The binary image outputs (cropped canvas snapshots) are saved directly to local disk storage (data_logs/interventions/), and the CSV files simply maintain the relative filepath pointers¹. By cross-referencing instances where users reported extreme difficulty with the granular stroke kinematics stored in these CSVs, future predictive models can be fine-tuned to predict student frustration before it visibly manifests on the canvas¹.

7.2 The TextTeller Fine-Tuning Pipeline

The primary purpose of accumulating the DSC_handwriting_dataset.csv is to resolve the system's over-reliance on pre-trained weights and synthetic datasets. Currently, fallback character disambiguation modules (e.g., stroke_symbol.py) are trained entirely on procedurally generated synthetic doodles—literal parametric ellipses standing in for digits—covering only a fraction of required symbols⁶.

By capturing true user ink alongside verified LaTeX ground truth from the TrainPlaygroundPage, the architecture establishes a direct, frictionless pipeline to invoke TextTeller's native training scripts⁵. The environment is provisioned with uv pip install textteller[train], installing all necessary dependencies for Hugging Face's Accelerate framework⁶.

The backend converts the CSV data into the precise metadata.jsonl format expected by the model⁶. During overnight downtime, the application can programmatically invoke accelerate launch train.py via the POST /api/math/train/retrain endpoint³. This allows the system to actively fine-tune the Vision Transformer on the user's specific handwriting profile, continually increasing OCR accuracy without ever transmitting personal biometric or cognitive data to the cloud⁶.

By synthesizing high-frequency DOM manipulation techniques, lightweight OpenCV pre-processing, dynamic procedural question generation, and rigorously constrained local LLM execution, this architecture establishes a highly responsive, closed-loop educational framework. It eliminates the latency and privacy vulnerabilities inherent to cloud-based APIs while operating flawlessly within the strict computational confines of consumer-grade edge hardware.

Works cited

1. AI Math Tutor Technical Deep Dive, https://drive.google.com/open?id=1L16zHnZzGi-TAdkYf3m-1_NrU7qZvO86vyXQlqkOUFU
2. Local Math OCR Tutor Architecture,

- https://drive.google.com/open?id=1aaAJGkILf_hsB0sAHAU9pQvPn899RxRFVltJe5N-GOc
3. router.py,
https://drive.google.com/open?id=1XSax0Zkls_T2vyoxqsC1FZ3K5hGas-xK
 4. LLD.md,
<https://drive.google.com/open?id=1Futy4apvfaY6PdNGYH2fQ-DmcbqYbSWT>
 5. FILE_INDEX.md,
https://drive.google.com/open?id=1R8LQlnMXvVKDwfY_16tnAdtEC7pa7jVj
 6. intervention_handler.py,
https://drive.google.com/open?id=1yTDx9Q6_VFzDCMhM953UNwBm9xIQPhb1
 7. MATH_OCR_ARCHITECTURE_EXPORT.md,
https://drive.google.com/open?id=1w-nMgSC1SjYrb8yc0aaUhq_-NCfhxeMq
 8. Math OCR Model Development Strategy,
<https://drive.google.com/open?id=1TjjAxZ6bFL9nZ4DfBLtb17PaeXjsKAViM06FadP1jAE>
 9. Pix2Text: The Markdown Converter Every Developer Needs,
<https://www.blog.brightcoding.dev/2026/03/13/pix2text-the-revolutionary-markdown-converter-every-developer-needs>
 10. Albumentations on GitHub in Documents and OCR,
<https://albumentations.ai/adoption/verticals/documents-ocr/github/>
 11. the latency of the fp16 onnx model is longer than that for the fp32,
<https://github.com/microsoft/onnxruntime/issues/8637>
 12. Deploying ONNX Models Made Easy: A Practical Step-by-Step Tutorial,
<https://ml-digest.com/deploying-onnx-models/>
 13. ONNX Runtime Performance Tuning,
<https://oliviajain.github.io/onnxruntime/docs/performance/tune-performance.html>
 14. ONNX Runtime Performance Tuning - Gitee,
https://gitee.com/loic_wang/onnxruntime/blob/master/docs/ONNX_Runtime_Perf_Tuning.md
 15. [Performance] ONNX Runtime GPT2 Model Running Significantly,
<https://github.com/microsoft/onnxruntime/issues/13105>
 16. Building MLOps Infrastructure for Trading AI Models — TRUETECH,
<https://truetechnology.dev/ai-development/services/mlops/mlops-infrastructure-trading-ai-models.html>
 17. ONNX Runtime Performance Tuning - GitHub,
https://github.com/yuhc/ava-onnxruntime/blob/r1.2.0/docs/ONNX_Runtime_Perf_Tuning.md
 18. parse_latex is bound to a specific version of antlr4 #27026 - GitHub,
<https://github.com/sympy/sympy/issues/27026>
 19. sympy.parsing.latex.parse_latex raising an error with antlr4-python3,
<https://github.com/sympy/sympy/issues/23355>
 20. Parsing - SymPy 1.14.0 documentation,
<https://docs.sympy.org/latest/modules/parsing.html>
 21. Getting an TypeError when trying to use parse_latex() - Stack Overflow,
<https://stackoverflow.com/questions/79883275/getting-an-typeerror-when-trying>

[-to-use-parse-latex](#)

22. parse_latex should not use evaluate=False by default #24116 - GitHub,
<https://github.com/sympy/sympy/issues/24116>
23. TexTeller: 让数学公式从图片中“活”起来的AI神器原创 - CSDN博客,
https://blog.csdn.net/gitblog_00896/article/details/160851730
24. TexTeller公式识别技术深度剖析: 从8000万数据训练到生产级部署,
https://blog.csdn.net/gitblog_00215/article/details/160852242
25. TexTeller can convert image to latex formulas (image2latex ... - GitHub,
<https://github.com/OleehyO/TeXTeller>
26. 如何用8000万数据训练出超越传统OCR的数学公式转换神器-CSDN博客,
https://blog.csdn.net/gitblog_00777/article/details/160822386